

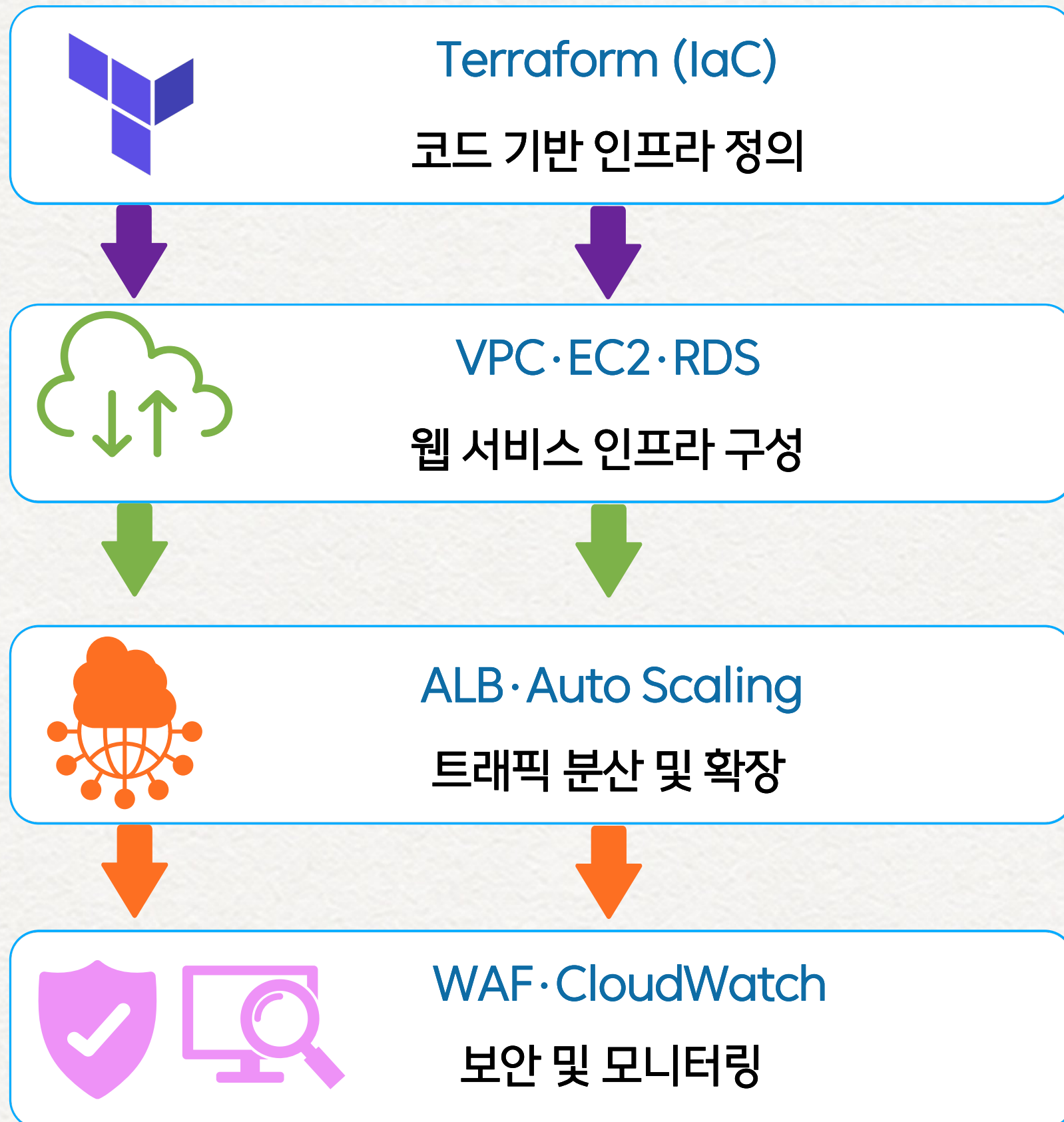


Terraform 기반 AWS 인프라 구축 Portfolio

Infrastructure · IaC · Cloud · Security

Terraform 기반 AWS 웹 서비스 인프라 구축

(Terraform | AWS | IaC)



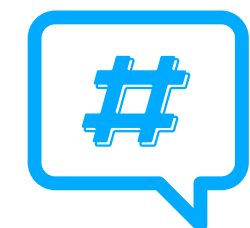
프로젝트 목적

코드 기반으로 재현 가능한 AWS 웹 서비스 인프라를 구축하고, 트래픽 대응 · 보안 · 모니터링까지 고려한 안정적인 운영 환경을 구현하는 것



프로젝트 기간

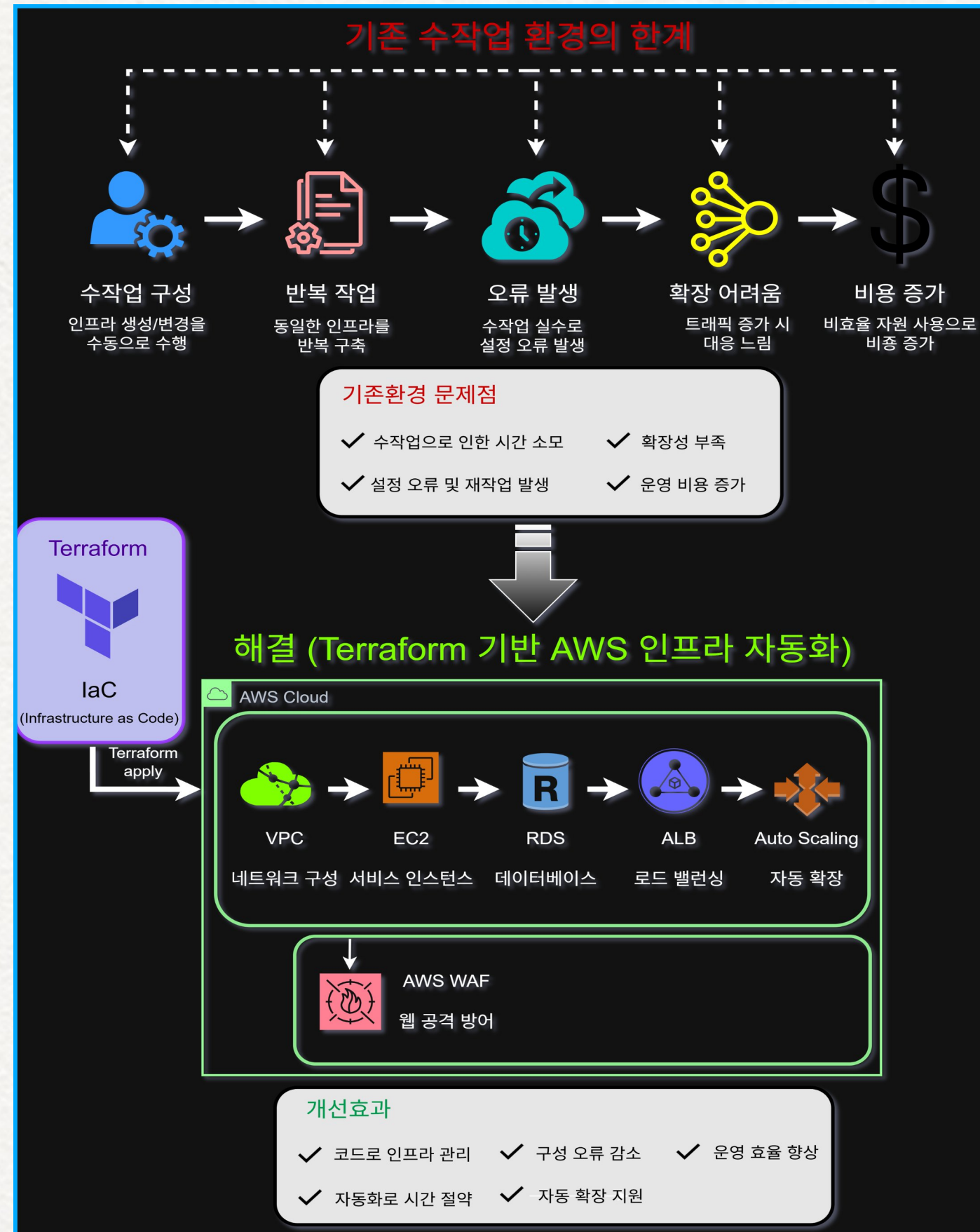
2025.12.17 ~ 2026.01.02



핵심 키워드

#Terraform #AWS #IaC
#Web Service #Auto Scaling
#WAF #CloudWatch

Cloud (AWS)



문제 인식

수작업 기반 AWS 인프라 구축의 한계

기존 AWS 환경에서는 리소스를 개별적으로 생성·설정하여 반복 작업과 설정 오류 가능성이 있었고, 동일 환경의 재현 및 변경 관리에도 어려움이 있었습니다.

해결 방향

Terraform 기반 IaC 적용

AWS 리소스를 Terraform 코드로 정의하여 동일한 인프라를 재현하고 변경 사항을 일관되게 관리하고자 했습니다.

수작업 구축

반복 작업 · 설정 오류 가능성 · 낮은 재현성 · 변경 관리 부담



Terraform IaC 적용

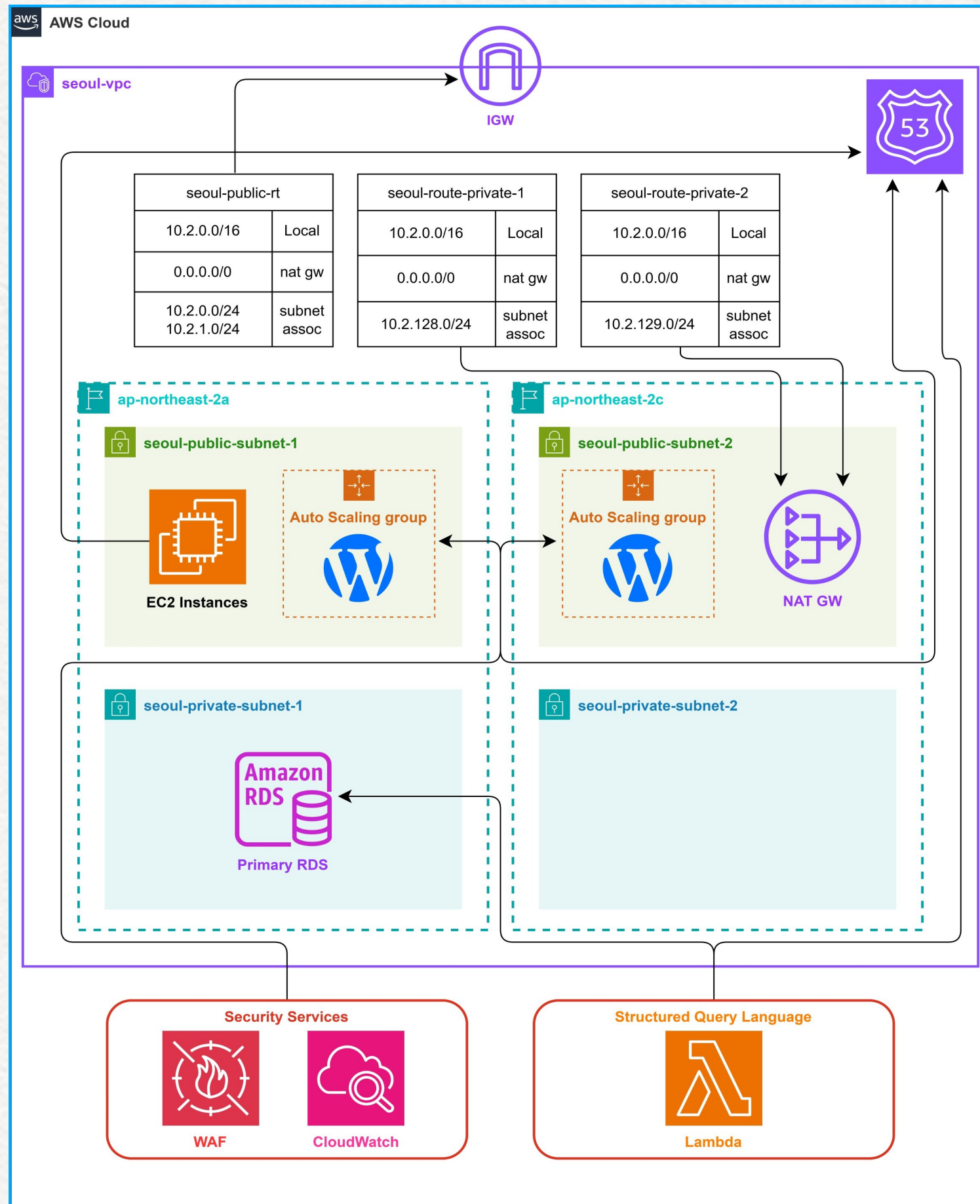
코드 기반 인프라 구축

재현성 · 일관성 · 자동화 · 변경 관리

구축 아키텍처

(Terraform 기반 AWS 웹 서비스 인프라 구성)

AWS Architecture



아키텍처 구성

01. 네트워크 영역 분리

Public · Private Subnet을 분리하여 웹 서비스와 데이터베이스 영역 구성

02. 웹 서비스 및 트래픽 처리

EC2 · ALB · Auto Scaling을 연계하여 트래픽 분산 및 자동 확장 구성

03. 데이터베이스 구성

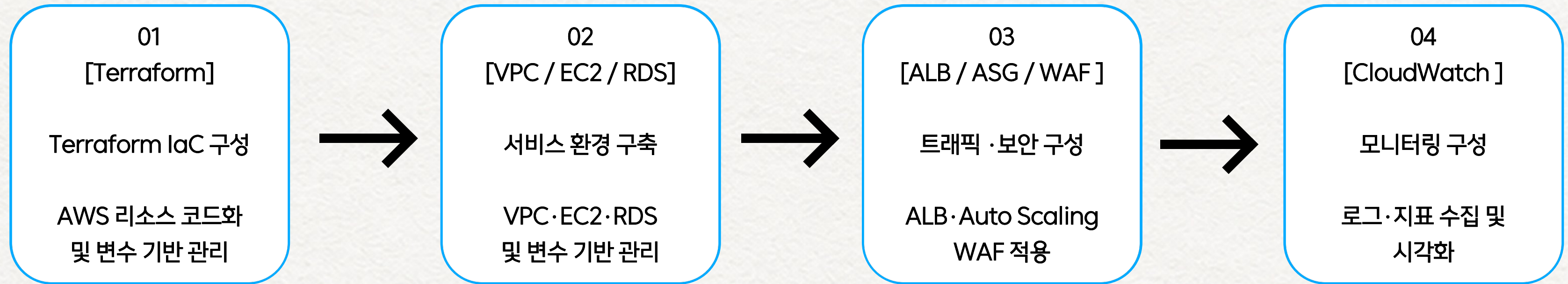
RDS를 Private Subnet에 배치하여 외부 직접 접근 제한

04. 보안 및 모니터링

AWS WAF를 통한 웹 요청 보호 및 CloudWatch 기반 모니터링 구성

Terraform으로 주요 AWS 리소스를 코드화하여 동일한 인프라 구성을 재현할 수 있도록 설계했습니다.

Cloud 기반 IaC 구축 과정



IaC 구현
[Terraform 코드]

```
terraform {  
  required_version = "~> 1.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}  
  
locals {  
  conf = yamldecode(file("./variables.yaml"))  
}  
  
provider "aws" {  
  region = local.conf.region  
}
```

모니터링 결과
[CloudWatch Dashboard]





CloudWatch Dashboard

```
=====
WAF Attack Test Started
Target: http://www.siu.it-edu.org
=====

[1] SQL Injection Attack Test
Attack: http://www.siu.it-edu.org?id=1' OR '1'='1
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org?id=1' UNION SELECT NULL--
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org?id=1; DROP TABLE users--
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org?id=1' OR 1=1--
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org/wp-login.php?user=admin'--
[OK] Blocked by WAF (403 Forbidden)

[2] WordPress Specific Attack Test
Attack: http://www.siu.it-edu.org/wp-admin/admin-ajax.php?action=revslider_show_image&img=../wp-config.php
[OK] Blocked by WAF (403 Forbidden)

[3] Rate-based Brute Force Attack Test
Path: /wp-login.php (Blocked if 15+ requests in 5 min)
Sending 16 requests quickly...
Request #1... Allowed (HTTP 200)
Request #2... Allowed (HTTP 200)
Request #3... Allowed (HTTP 200)
Request #4... Allowed (HTTP 200)
Request #5... Allowed (HTTP 200)
Request #6... Allowed (HTTP 200)
Request #7... Allowed (HTTP 200)
Request #8... Allowed (HTTP 200)
Request #9... Allowed (HTTP 200)
Request #10... Allowed (HTTP 200)
Request #11... Allowed (HTTP 200)
Request #12... Allowed (HTTP 200)
Request #13... Allowed (HTTP 200)
Request #14... Allowed (HTTP 200)
Request #15... Allowed (HTTP 200)
Request #16... Allowed (HTTP 200)

Result: Allowed 16 times, Blocked 0 times

=====
WAF Attack Test Completed
=====
```

WAF Attack Test



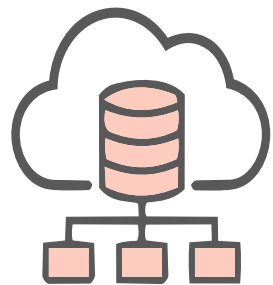
프로젝트 성과

- 01. 코드 기반 인프라 구축
Terraform을 활용해 AWS 리소스를 코드로 정의하고, 동일한 인프라 구성을 반복적으로 재현할 수 있는 환경을 구현했습니다.
- 02. 트래픽 대응 및 웹 보안 구성
ALB와 Auto Scaling을 연계하여 트래픽 분산 및 자동 확장 구조를 구성하고, AWS WAF를 적용하여 웹 공격 차단 동작을 검증했습니다.
- 03. 모니터링 환경 구현
CloudWatch Dashboard를 구성하여 웹 서비스의 로그와 주요 지표를 확인할 수 있는 모니터링 환경을 구현했습니다.



1 IaC의 목적과 운영 관점

Terraform을 활용해 AWS 리소스를 코드로 정의하며, IaC는 단순히 구축 작업을 자동화하는 것이 아니라 동일한 환경을 재현하고 변경 사항을 일관되게 관리하기 위한 방식이라는 점을 경험했습니다.



2 서비스 운영을 고려한 인프라 설계

VPC, EC2, RDS, ALB, Auto Scaling 등을 연계하며 개별 리소스의 구성에 그치지 않고 서비스의 트래픽 흐름과 네트워크·데이터베이스 구조를 함께 고려하여 인프라를 설계하는 것이 중요하다는 점을 배웠습니다.



3 보안과 모니터링까지 고려한 구축

AWS WAF와 CloudWatch를 구성하고 실제 동작을 확인하면서, 인프라는 서비스 구축뿐만 아니라 보안 정책 적용과 운영 상태를 지속적으로 확인할 수 있는 환경까지 함께 고려해야 한다는 점을 배웠습니다.